

```
# Q1
def compute_division():
    try:
        result = 5 / 0
    except ZeroDivisionError as e:
        print(f"Error caught: {e}")

compute_division()
```

Error caught: division by zero

```
# Q2
def access_array():
    data = [10, 20, 30]
    try:
        print(data[5])
    except IndexError as e:
        print(f"Index Error: {e}")

access_array()
```

Index Error: list index out of range

```
# Q3
def open_and_read_file():
    file_name = input("Enter file name: ")
    try:
        with open(file_name, 'r') as f:
            content = f.read()
            number = float(content)
            print(f"File content as number: {number}")
    except FileNotFoundError:
        print("Error: The file does not exist.")
    except ValueError:
        print("Error: File contains non-numeric data.")

open_and_read_file()
```

Enter file name: 3456
Error: The file does not exist.

```
# Q4
class InvalidAgeError(Exception):
    pass

def check_age():
    try:
        age = int(input("Enter your age: "))
        if age < 0 or age > 150:
            raise InvalidAgeError(f"Age {age} is out of valid range (0-150).")
        print(f"Valid age: {age}")
    except InvalidAgeError as e:
        print(e)
    except ValueError:
        print("Please enter a valid integer.")

check_age()
```

Enter your age: 29
Valid age: 29

```
# Q5
def divide_with_finally():
    try:
        a = float(input("Enter numerator: "))
        b = float(input("Enter denominator: "))
        result = a / b
    except ZeroDivisionError:
        print("Error: Cannot divide by zero.")
    else:
        print(f"Result is: {result}")
    finally:
        print("Operation Complete.")
```

```
divide_with_finally()
```

```
Enter numerator: 45
Enter denominator: 5
Result is: 9.0
Operation Complete.
```

```
# Q6
def convert_to_int(val):
    try:
        result = int(val)
        print(f"Converted: {result}")
    except ValueError:
        print("Error: Not a valid integer string.")
    except TypeError:
        print("Error: Input is an empty string or incompatible type.")

convert_to_int("")
```

```
Error: Not a valid integer string.
```

```
# Q7
def inner_function():
    print("Inner function: Raising ValueError...")
    raise ValueError("The original source of the problem.")

def outer_function():
    try:
        inner_function()
    except ValueError as original_error:
        print("Outer function: Caught ValueError, now raising RuntimeError with more info.")

        raise RuntimeError("Additional information: Failed to complete the task.") from original_error

print("Testing Exception Chaining")
try:
    outer_function()
except RuntimeError as final_error:
    print(f"\nFinal Caught Error: {final_error}")
    print(f"Original Cause: {final_error.__cause__}")
```

```
Testing Exception Chaining
Inner function: Raising ValueError...
Outer function: Caught ValueError, now raising RuntimeError with more info.

Final Caught Error: Additional information: Failed to complete the task.
Original Cause: The original source of the problem.
```

```
# Q8
def handle_multiple():
    try:

        my_list = [1, 2]
        my_dict = {"a": 1}

        print(my_dict["z"])

    except (IndexError, KeyError) as e:
        print(f"Caught a collection-related error: {type(e).__name__} - {e}")

handle_multiple()
```

```
Caught a collection-related error: KeyError - 'z'
```

```
# Q9
def process_file_lines(lines):
    for line in lines:
        try:

            try:
                num = int(line.strip())
                print(f"Processed: {num}")
            except ValueError:
```

```
        print(f"Skipping non-numeric line: '{line.strip()}'")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

process_file_lines(["10", "abc", "20"])
```

```
Processed: 10
Skipping non-numeric line: 'abc'
Processed: 20
```

```
# Q10
def test_readonly_error():
    filename = "test_file.txt"

    with open(filename, "w") as f:
        f.write("Initial content.\n")

    print(f"testing read-only")
    try:

        with open(filename, "r") as f:
            print("File opened in read-only mode. Attempting to write...")
            f.write("This should fail.")
    except Exception as e:
        print(f"Caught Error: {e}")
    finally:
        print("Finished Q10 check.\n")

test_readonly_error()
```

```
testing read-only
File opened in read-only mode. Attempting to write...
Caught Error: not writable
Finished Q10 check.
```

```
# Q11
def check_positive_value(x):
    print(f"Checking if {x} is lower than 0")
    try:
        if x < 0:

            raise ValueError(f"Stop! The value {x} is lower than 0.")
        else:
            print(f"Success: {x} is a valid positive number.")
    except ValueError as e:
        print(f"Program stopped by error: {e}")

check_positive_value(-5)
print("")
```

```
Checking if -5 is lower than 0
Program stopped by error: Stop! The value -5 is lower than 0.
```

```
# Q12
def ensure_integer(x):
    print(f"Type Validation for '{x}'")
    try:
        if not isinstance(x, int):

            raise TypeError(f"Only integers are allowed. You provided a {type(x).__name__}.")
        else:
            print(f"Success: {x} is an integer.")
    except TypeError as e:
        print(f"Caught Type Error: {e}")

ensure_integer("10")
```

```
Type Validation for '10'
Caught Type Error: Only integers are allowed. You provided a str.
```

